📘 Dictionaries in Python

Dictionary Operations | All Built-in Dictionary Methods/Functions

Introduction to Dictionaries

Definition: A dictionary is an unordered collection of key-value pairs enclosed in curly braces (`{ }`), where each key is unique and maps to a value.

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
dict2 = {1: "one", 2: "two", 3: "three"}
dict3 = {}                          # empty dictionary
dict4 = dict(name="Bob", age=30) # using dict() constructor
```

Important Points:

- Dictionaries are mutable (can be changed after creation)
- Keys must be unique and immutable (string, number, tuple)
- Values can be of any data type and can be duplicated
- Dictionaries are unordered (Python 3.7+ maintains insertion order)
- Accessing elements is done using keys, not index positions

Accessing Items in a Dictionary Using Keys

1. Using Square Brackets [ ]

Purpose: Accesses the value associated with a key

Syntax: `dictionary[key]`

Parameters: key (compulsory) — the key whose value is needed

Return: The value associated with the key

- Raises `KeyError` if the key does not exist
- Fastest way to access a value

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(dict1["name"])   # "Alice"
```

```
print(dict1["age"])    # 25
# print(dict1["country"])  # KeyError
```

2. Using get() Method

Purpose: Accesses the value associated with a key safely

Syntax: `dictionary.get(key, default)`

Parameters:

- key (compulsory) — the key whose value is needed
- default (optional) — value to return if key not found (default: None)

Return: The value associated with the key, or default if not found

- Does not raise KeyError if key is missing
- Safer alternative to square bracket access

```
dict1 = {"name": "Alice", "age": 25}
print(dict1.get("name"))            # "Alice"
print(dict1.get("country"))         # None
print(dict1.get("country", "India")) # "India"
```

Mutability of a Dictionary

1. Adding a New Item

Purpose: Adds a new key-value pair to the dictionary

Syntax: `dictionary[new_key] = value`

Parameters: new_key (compulsory) — key to add; value (compulsory) — value to assign

Return: None (modifies the original dictionary)

- If the key already exists, its value is updated instead
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25}
dict1["city"] = "Delhi"
print(dict1)  # {"name": "Alice", "age": 25, "city": "Delhi"}
```

2. Modifying an Existing Item

Purpose: Changes the value associated with an existing key

Syntax: `dictionary[existing_key] = new_value`

Parameters:

existing_key (compulsory) — key to modify; new_value (compulsory) — new value

Return: None (modifies the original dictionary)

- If the key exists, its value is replaced
- If the key does not exist, a new key-value pair is added
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25}
dict1["age"] = 26
print(dict1)  # {"name": "Alice", "age": 26}
```

3. Deleting an Item Using del

Purpose: Removes a key-value pair from the dictionary

Syntax: `del dictionary[key]`

Parameters: key (compulsory) — key to be removed

Return: None (modifies the original dictionary)

- Raises `KeyError` if the key does not exist
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
del dict1["age"]
print(dict1)  # {"name": "Alice", "city": "Delhi"}
```

Traversing a Dictionary

Using for loop (keys only):

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
for key in dict1:
```

```
    print(key)
# Output: name  age  city (each on new line)
```

Using for loop (keys and values):

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
for key in dict1:
    print(key, ":", dict1[key])
```

Using items() method:

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
for key, value in dict1.items():
    print(key, ":", value)
```

Using values() method:

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
for value in dict1.values():
    print(value)
```

Built-in Functions / Methods

1. len()

Purpose: Returns the number of key-value pairs in a dictionary

Syntax: `len(dictionary)`

Parameters: dictionary (compulsory)

Return: Integer (number of items)

- Counts all key-value pairs

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(len(dict1))  # 3
```

2. dict()

Purpose: Creates a new dictionary

Syntax: `dict(iterable)` or `dict(**kwargs)`

Parameters: iterable (optional) — sequence of key-value pairs

Return: A new dictionary

- If no argument given, returns an empty dictionary
- Can create from list of tuples or keyword arguments

```
print(dict())                           # {}
print(dict([(1, "one"), (2, "two")]))   # {1: "one", 2: "two"}
print(dict(name="Alice", age=25))       # {"name": "Alice", "age": 25}
```

3. keys()

Purpose: Returns all the keys in the dictionary

Syntax: `dictionary.keys()`

Parameters: No parameters

Return: A view object containing all keys

- Returns a dynamic view (reflects changes to the dictionary)
- Can be converted to a list using `list()`

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(dict1.keys())          # dict_keys(["name", "age", "city"])
print(list(dict1.keys()))    # ["name", "age", "city"]
```

4. values()

Purpose: Returns all the values in the dictionary

Syntax: `dictionary.values()`

Parameters: No parameters

Return: A view object containing all values

- Returns a dynamic view (reflects changes to the dictionary)
- Can be converted to a list using `list()`

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(dict1.values())          # dict_values(["Alice", 25, "Delhi"])
print(list(dict1.values()))    # ["Alice", 25, "Delhi"]
```

5. items()

Purpose: Returns all key-value pairs as tuples

Syntax: `dictionary.items()`

Parameters: No parameters

Return: A view object containing all key-value pairs as tuples

- Returns a dynamic view (reflects changes to the dictionary)
- Useful for iterating with both key and value
- Can be converted to a list using `list()`

```
dict1 = {"name": "Alice", "age": 25}
print(dict1.items())          # dict_items([("name", "Alice"), ("age", 25)])
print(list(dict1.items()))    # [("name", "Alice"), ("age", 25)]
```

6. get()

Purpose: Returns the value associated with a key safely

Syntax: `dictionary.get(key, default)`

Parameters:

- key (compulsory) — the key whose value is needed
- default (optional) — value to return if key not found (default: None)

Return: The value associated with the key, or default if not found

- Does not raise KeyError if key is missing
- Safer alternative to square bracket access

```
dict1 = {"name": "Alice", "age": 25}
print(dict1.get("name"))              # "Alice"
print(dict1.get("country"))           # None
print(dict1.get("country", "India")) # "India"
```

7. update()

Purpose: Updates the dictionary with key-value pairs from another dictionary or iterable

Syntax: `dictionary.update(other)`

Parameters: other (compulsory) — dictionary or iterable of key-value pairs

Return: None (modifies the original dictionary)

- Existing keys are overwritten with new values
- New keys are added to the dictionary
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25}
dict1.update({"age": 26, "city": "Delhi"})
print(dict1)  # {"name": "Alice", "age": 26, "city": "Delhi"}
```

8. del

Purpose: Removes a key-value pair or the entire dictionary

Syntax: `del dictionary[key]` or `del dictionary`

Parameters: key (optional) — key to be removed; if omitted, deletes the entire dictionary

Return: None (modifies or deletes the original dictionary)

- Raises `KeyError` if the key does not exist
- `del dictionary` removes the entire dictionary from memory

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
del dict1["age"]
print(dict1)  # {"name": "Alice", "city": "Delhi"}

del dict1
# print(dict1)  # NameError: name 'dict1' is not defined
```

9. clear()

Purpose: Removes all key-value pairs from the dictionary

Syntax: `dictionary.clear()`

Parameters: No parameters

Return: None (modifies the original dictionary)

- Empties the dictionary but keeps the dictionary object
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25}
dict1.clear()
print(dict1)  # {}
```

10. fromkeys()

Purpose: Creates a new dictionary with specified keys and a common value

Syntax: `dict.fromkeys(keys, value)`

Parameters:

- keys (compulsory) — iterable of keys
- value (optional) — common value for all keys (default: None)

Return: A new dictionary

- All keys are assigned the same value
- Does not modify the original dictionary

```
keys = ["a", "b", "c"]
dict1 = dict.fromkeys(keys)
print(dict1)  # {"a": None, "b": None, "c": None}

dict2 = dict.fromkeys(keys, 0)
print(dict2)  # {"a": 0, "b": 0, "c": 0}
```

11. copy()

Purpose: Returns a shallow copy of the dictionary

Syntax: `dictionary.copy()`

Parameters: No parameters

Return: A new dictionary with the same key-value pairs

- Creates a shallow copy (nested objects are shared)
- Modifying the copy does not affect the original

```
dict1 = {"name": "Alice", "age": 25}
dict2 = dict1.copy()
dict2["age"] = 26
print(dict1)  # {"name": "Alice", "age": 25}
print(dict2)  # {"name": "Alice", "age": 26}
```

12. pop()

Purpose: Removes and returns the value associated with a key

Syntax: `dictionary.pop(key, default)`

Parameters:

- key (compulsory) — key to be removed
- default (optional) — value to return if key not found

Return: The value associated with the removed key

- Raises `KeyError` if key not found and no default given
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(dict1.pop("age"))              # 25
print(dict1)                         # {"name": "Alice", "city": "Delhi"}
print(dict1.pop("country", "N/A"))   # "N/A"
```

13. popitem()

Purpose: Removes and returns the last inserted key-value pair

Syntax: `dictionary.popitem()`

Parameters: No parameters

Return: A tuple containing the removed key-value pair

- Removes the last inserted item (LIFO order)
- Raises `KeyError` if the dictionary is empty
- Modifies the original dictionary in place

```
dict1 = {"name": "Alice", "age": 25, "city": "Delhi"}
print(dict1.popitem())  # ("city", "Delhi")
print(dict1)            # {"name": "Alice", "age": 25}
```

14. setdefault()

Purpose: Returns the value of a key; if the key does not exist, inserts it with a default value

Syntax: `dictionary.setdefault(key, default)`

Parameters:

- key (compulsory) — key to search for
- default (optional) — value to set if key not found (default: None)

Return: The value associated with the key

- If key exists, returns its value without modifying the dictionary
- If key does not exist, adds it with the default value

```
dict1 = {"name": "Alice", "age": 25}
print(dict1.setdefault("age", 30))    # 25 (key exists)
print(dict1.setdefault("city", "Delhi")) # "Delhi" (key added)
print(dict1)  # {"name": "Alice", "age": 25, "city": "Delhi"}
```

15. max()

Purpose: Returns the largest key in the dictionary

Syntax: `max(dictionary)`

Parameters: dictionary (compulsory)

Return: The largest key

- Works on keys, not values
- Keys must be of comparable types
- Raises ValueError if dictionary is empty

```
dict1 = {3: "three", 1: "one", 2: "two"}
print(max(dict1))  # 3
```

16. min()

Purpose: Returns the smallest key in the dictionary

Syntax: `min(dictionary)`

Parameters: dictionary (compulsory)

Return: The smallest key

- Works on keys, not values
- Keys must be of comparable types
- Raises ValueError if dictionary is empty

```
dict1 = {3: "three", 1: "one", 2: "two"}
print(min(dict1))  # 1
```

17. sorted()

Purpose: Returns a new sorted list of the dictionary's keys

Syntax: `sorted(dictionary, reverse=True/False)`

Parameters:

- dictionary (compulsory) — the dictionary to be sorted
- reverse (optional) — if True, sorts in descending order (default: False)

Return: A new sorted list of keys

- Does not modify the original dictionary
- Returns a list, not a dictionary
- To sort by values, use `sorted(dict, key=dict.get)`

```
dict1 = {3: "three", 1: "one", 2: "two"}
print(sorted(dict1))              # [1, 2, 3]
print(sorted(dict1, reverse=True)) # [3, 2, 1]

# Sort by values
print(sorted(dict1, key=dict1.get)) # [1, 3, 2]
```

Suggested Programs

1. Count the Number of Times a Character Appears in a Given String Using a Dictionary

Purpose: Count the frequency of each character in a string

```
string = "hello world"

# Method 1: Using a dictionary
char_count = {}
for char in string:
    if char in char_count:
        char_count[char] += 1
    else:
        char_count[char] = 1

print(char_count)
# Output: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

2. Count Characters Using get() Method

```
string = "hello world"

char_count = {}
for char in string:
    char_count[char] = char_count.get(char, 0) + 1

print(char_count)
# Output: {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

3. Count Characters Excluding Spaces

```
string = "hello world"

char_count = {}
for char in string:
    if char != " ":
        char_count[char] = char_count.get(char, 0) + 1

print(char_count)
# Output: {'h': 1, 'e': 1, 'l': 3, 'o': 2, 'w': 1, 'r': 1, 'd': 1}
```

4. Create a Dictionary with Names of Employees, Their Salary, and Access Them

Purpose: Store employee names and salaries, then access individual salaries

```
# Create a dictionary of employees and their salaries
employees = {
    "Alice": 50000,
    "Bob": 60000,
    "Charlie": 55000,
    "Diana": 70000
}

# Access individual salaries
print("Alice's salary:", employees["Alice"])       # 50000
print("Bob's salary:", employees.get("Bob"))       # 60000
print("Charlie's salary:", employees["Charlie"])   # 55000

# Access using get() with default
print("Eve's salary:", employees.get("Eve", "Not Found"))  # "Not Found"
```

5. Display All Employees and Their Salaries

```
employees = {
    "Alice": 50000,
    "Bob": 60000,
    "Charlie": 55000,
    "Diana": 70000
}

# Traverse the dictionary
for name, salary in employees.items():
    print(name, "earns", salary)

# Output:
# Alice earns 50000
# Bob earns 60000
# Charlie earns 55000
# Diana earns 70000
```

6. Find the Highest Paid Employee

```
employees = {
    "Alice": 50000,
    "Bob": 60000,
    "Charlie": 55000,
    "Diana": 70000
}

highest_paid = max(employees, key=employees.get)
print("Highest paid employee:", highest_paid)  # "Diana"
print("Salary:", employees[highest_paid])      # 70000
```

7. Add a New Employee and Update an Existing Salary

```
employees = {
    "Alice": 50000,
    "Bob": 60000
}

# Add a new employee
employees["Charlie"] = 55000
print(employees)  # {"Alice": 50000, "Bob": 60000, "Charlie": 55000}

# Update an existing salary
employees["Alice"] = 52000
print(employees)  # {"Alice": 52000, "Bob": 60000, "Charlie": 55000}
```

Quick Reference Table

Method / Function	Purpose	Returns
`len()`	Number of items	Integer
`dict()`	Create a dictionary	Dictionary
`keys()`	All keys	View object
`values()`	All values	View object

Method / Function	Purpose	Returns
`items()`	All key-value pairs	View object
`get()`	Safe access to value	Value / Default
`update()`	Merge dictionaries	None
`del`	Delete item/dictionary	None
`clear()`	Remove all items	None
`fromkeys()`	Create dict from keys	Dictionary
`copy()`	Shallow copy	Dictionary
`pop()`	Remove and return value	Value
`popitem()`	Remove and return last item	Tuple
`setdefault()`	Get or insert default	Value
`max()`	Largest key	Key
`min()`	Smallest key	Key
`sorted()`	Sorted list of keys	List

Important Board Exam Tips Must Know

1. **Dictionary Mutability:** Dictionaries are mutable. You can add, modify, or delete key-value pairs after creation.
2. **Keys Must Be Unique:** Duplicate keys are not allowed. If a key is repeated, the last value overwrites the previous one.
3. **Keys Must Be Immutable:** Keys can be strings, numbers, or tuples, but not lists or dictionaries.
4. **Accessing Items:**
   - `dict[key]` : Raises KeyError if key not found
   - `dict.get(key, default)` : Returns default if key not found (safer)
5. **Traversing a Dictionary:**
   - `for key in dict:` — iterates over keys
   - `for key, value in dict.items():` — iterates over key-value pairs

- `for value in dict.values():` — iterates over values

6. **Adding vs Modifying:**
   - If the key exists, `dict[key] = value` modifies the value
   - If the key does not exist, `dict[key] = value` adds a new item
7. **pop() vs popitem() vs del:**
   - `pop(key)` : Removes and returns value for a specific key
   - `popitem()` : Removes and returns the last inserted key-value pair
   - `del dict[key]` : Deletes a key-value pair without returning
   - `del dict` : Deletes the entire dictionary
8. **clear() vs del:**
   - `clear()` : Empties the dictionary but keeps the object
   - `del dict` : Removes the dictionary object entirely
9. **sorted() on Dictionary:**
   - `sorted(dict)` returns a sorted list of keys
   - `sorted(dict, key=dict.get)` sorts by values
10. **Common Errors to Avoid:**
    - Using a mutable key (like a list) — raises TypeError
    - Accessing a non-existent key with [ ] — raises KeyError
    - Confusing `keys()` , `values()` , and `items()`
    - Forgetting that `popitem()` removes the last inserted item (Python 3.7+)
    - Using `max()` / `min()` on dictionaries with non-comparable keys

---